1) solve the following problem with CPU Scheduling Algorithms

Draw a table with 4 rows with

Process	AT	BT
P_1	0	5
P_2	0	4
P_3	0	3

If there is a context switch in the CPU scheduling Algorithm then there is a context switch time of 5 sec so each unit is considered as seconds.

Two Scheduling Algorithms

1) Preemptive

2) Non-Preemptive.

Once CPU is assigned with anyone of the Process

opp of preemptive

3 Scheduling Algorithms

1) SRTF shortest Run Time first

2) PS

3) RR (round robbing)

Same.

1) FCFS (First come first serve)

2) SJF (Shortest Jaw first)

3) PS (Priority scheduling)

for given problems

If 4 columns have we can apply all 6 algorithms

$\underbrace{\text{(Process, AT, BT, Priority)}}_{\text{4 columns}}$

If last column is not there we can only use 4 algorithms (PS x)

(Process, AT, BT)

if TQ is not given, we have to assume as 1

FCFS:

- Arrival of process
- All three process are arrived at same time that is 0.

RQ → Data Structure shows process which are available for execution.

GT

RQ

0: | P_1 | P_2 | P_3 |

GT

| | | | | - - - | |
0 1 2 3 4 n

If all arrive at same then consider least bus time

also

0: | P_1 | P_2 | P_3 |

| P_3 | P_1 | P_2 |

because all three arrived at same time

GT

0: | P_1 | P_2 | P_3 |

frontend (remove)

Rare end (Adding)

| | | | | - - - | |
0 1 2 3 4 n

| | | |
0 1 2 3 4 5 n

If we apply CPU to process it is remained with process until the process execution is completed (Non-preemptive)

- Amount of Time required for completion of process execution is Called Bus Time.

5: | ~~P_1~~ | P_2 | P_3 |

| | P_2 | P_3 |
0 1 2

frontend = 1

Rare index = 2

GT

| P_1 | P_2 | |
0 1 2 3 4 5 - - - 9

4 units required for P_2

9: | | ~~P_2~~ | P_3 |

F = 1 0+1 = 2. T = 2

GT

| P_1 | P_2 | P_3 |
0 1 2 3 4 5 - - - 9 - - - 12

f = 2 = R = 2 so the list / queue is having one element

| | | P_3 |

f = R = equals -1 ⇒ queue empty

P_3 is completed at 12 second

P_3 is removed there

P_2: | | | ~~P_3~~ |

f = 2   R = 2

now f = R = -1

queue is empty

when queue is empty, 2 situations might occur:

1) All process are completed

2) During that time CPU

Using GT calculate CP completion Time, TAT, WT & RT Response time.

Turn Around Time	Waiting time CT	TAT CT-AT	WT TAT-BT	RT
P_1	5	5	0	0
P_2	9	9	5	5
P_3	12	12	9	9

When CPU is assigned for RT

for Non-preemptive, WT & RT are same.

<u>CPU length</u>:

Ending position of CPU - Starting position of CPU

12 - 0 = 12

Throughput:

No. of process executed / length = 3/12 = 0.25

per unit 0.25 no. of process are executed.

maximizing Throughput & minimizing WT & RT

Avg = 26/3 $\dfrac{\text{Sum of TAT}}{\text{No. of process}}$

$= \dfrac{\text{Sum of WT}}{\text{No. of process}} = 14/3$

⇒ Is convation & starvation are applied to it or not.

⇒ Is content switeling is happening or not.

Which gives highest throughput & lowest Avg TAT & WT we can choose suitable Algorithm.

SRTF:

Shortest Remaining Time First

context switch:

RQ (shortest BT) GT

0: | P3 | P2 | P1 | | P3 | (0 1)

1: | P3 | P2 | P1 | executed by 1 unit out of 3 remaining = 2

acc tor ReT update RQ ReT(P3) = BT - current execution Time(P3)

3 - 1 = 2

1: | P3 | P2 | P1 | ReT(P2) = 4 - 0 = 4

no updation ReT(P1) = 5 - 0 = 5

GT

2: | P3 | P2 | P1 | (1 4 5) | P3 | P3 | (0 1 2)

ReT (P1) = 1

ReT (P2) = 4

ReT (P3) = 5

again select shortest RT that is P_3

3: | ~~P3~~ | P2 | P1 |
|---|---|---|

GT

P3	P3	P3	

0 1 2 3

ReT (P3) = 0 ⇒ Process execution is completed

4:

GT

P3	P3	P3	P2	P2	P2	P2	

0 1 2 3 4 5 6 7

at 7 P2 completed

GT

P3	P3	P3	P2	P2	P2	P2	P1 ---- 5times

0 1 2 3 4 5 6 7 12

Process	CT	TAT (CT-AT)	WT (TAT-BT)	RT
P1	12	12	7	
P2	7	7	3	
P3	3	3	0	

FCFS & SRTF

SRTF is best because giving less WT.

10) Real-life CPU scheduling scenario!

A mobile operating system has following tasks

Task	AT	CPU Burst
Music Player	0	20
Touch Event	3	1
Whatsapp Notification	5	2
Camera Launch	6	4

Questions:

1) Which scheduling algorithm is most suitable: FCFS, SJF, SRTF or RR?

2) Draw the Execution timeline for SRTF

3) Explain why the selected algorithm improve user experience.

4) Which task receives the fastest response time?

Ans

1) Suitable algorithm is SRTF because always checks the if a newly arrived task has a shorter remaining execution time than currently running task.

2) Execution timeline for SRTF is

at Time 0: m= music player

AT = 3

GT

m-- -	

0 3

Ret (mp) = 20-3 = 17

Bus time - current Execution Time

at Time 3:

T= Touch event arrives at 3. BT = 1

$ReT(M) = 17$

BT of TE(1)

$TE < Re(M)$

TE runs from 3 to 4

at Time 4:

Touch Event finishes

GT

M---	T	
0	3	4

$Re(T) = 17 - 1 = 16$

at Time 5:

Whatsapp Notification

arrival Time = 5

BT = 2

$2 < 16$

next AT = 6

at 6 WN has run for 1unit

Re(WN) = 1 Camera launch arrives

BT = 4

Now compare

Re(MP)(16) vs Re(WN)(1) vs CL(4)

Continue with WN

WN runs from 5 to 7

GT

M	T	WN	
0	3	4	7

at time 7 (WN) finishes

Now compare Re(M) 16 vs CL (4)

CL is shorter so allocating CPU to CL

CL runs from 7 to 11

at time 11 CL finishes

now allocating CPU to MP

MP runs continuosly until completion

11+16 = 27

GT

M	T	WN	CL	M

0 3 4 7 11 27

0 3 4 5 7 11 26

M	T	M	WN	CL	M

MP → TE → WN → CL → MP

3) SRTF give a quick response to small task like touch events and notifications. so, the mobile feels fast and responsive.

4) Touch

Task	AT	BT	first CPU	CT	TAT CT-AT	WT TAT-BT	RT
MP	0	20	0	27	27	7	0
TE	3	1	3	4	1	0	0
WN	5	2	5	7	2	0	0
CL	6	4	7	11	5	1	1

Music player, Touch Event & Whatsapp Notification Share the fastest response time of 0.

3) Define the following terms:,

Starvation, Convoy effect, Context switching, Aging.

1) Starvation: It occurs when a process waits indefinitely or for an excessively long time because other processes are repeatedly selected for execution. Starvation can occur in both preemptive & Non-preemptive scheduling, depending on the algorithm.

Ex: A low priority process may have to keep waiting while high priority process are continuosly executed.

2) Convoy Effect: It is an important problem associated with non-preemptive scheduling, especially FCFS. The long process acts like a convoy, causing many short processes to follow behind it.

3) Context Switching:

The PCB stores the current execution state of a process, inculding the program counter, CPU registers, stack pointer & other CPU related information when the CPU switches from one process to another, the OS saves the state of currently running process in its PCB & loads the saved state of the next process from its PCB. This allows the interrupted process to resume execution from exactly where it was paused.

- Aging: It is a technique used to prevent starvation by gradually increasing ~~us~~ the priority of a process that has been waiting for a long time.

A)

Process	AT	BT	Priority
P1	0	4	3
P2	0	9	1
P3	0	6	2
P4	0	4	3
P5	0	9	1
P6	0	6	2
P7	1	4	3
P8	1	9	1
P9	2	6	2

Apply all the CPU scheduling algorithms for insem.

FCFS : First come first serve

RQ 0: | P1 | P2 | P3 | P4 | P5 | P6 |

1: | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 |

2: | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 |

4: | ~~P1~~ | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 |

13: | ~~P2~~ | P3 | P4 | P5 | P6 | P7 | P8 | P9 | |

19: | ~~P3~~ | P4 | P5 | P6 | P7 | P8 | P9 | |

23: | ~~P4~~ | P5 | P6 | P7 | P8 | P9 |

32: | ~~P5~~ | P6 | P7 | P8 | P9 |

38: | ~~P6~~ | P7 | P8 | P9 |

42: | ~~P7~~ | P8 | P9 |

51: | ~~P8~~ | |

57: | ~~P9~~ |

GT:

P1	P2	P3	P4	P5	P6	P7	P8	P9

0 4 13 19 23 32 38 42 51 57

Process	AT	BT	CT	TAT CT-AT	WT TAT-BT	RT WT-AT
P_1	0	4	4	4	0	0
P_2	0	9	13	13	4	4
P_3	0	6	19	19	7	7
P_4	0	4	23	23	19	19
P_5	0	9	32	32	23	23
P_6	0	6	38	38	32	32
P_7	1	4	42	42	37	36
P_8	1	9	51	50	41	40
P_9	2	6	57	55	49	47.